Final Project Report: Loan Default Prediction

Course: EECS 4412 - Data Mining
Term: Fall 2025-2026
Date: December 2, 2025

Submitted by:

- Devarsh Prajapati (dev053@my.yorku.ca, 219899343)
- Pratham Khatri (Khatri26@my.yorku.ca, 219447424)
- Vedansh Patel (vedansh@my.yorku.ca, 219473990)
- Neel Shethji (nshethji@my.yorku.ca, 219381128)
- Rithik Chaudhary (rk10@my.yorku.ca, 219897750)

1. Introduction

1.1 Context & Motivation

Financial institutions face substantial risks from loan defaults, costing billions of dollars annually in the United States alone. Accurate default prediction enables lenders to:

- Make informed approval decisions and adjust interest rates based on risk.
- Maintain portfolio stability through healthy ratios of performing to non-performing loans.
- Comply with regulatory requirements (Basel III capital reserves based on risk profiles).
- Implement early intervention strategies for at-risk borrowers.

This project addresses the challenge of imbalanced classification in real-world financial data, where defaults (24.64%) are naturally rarer than successful repayments (75.36%). Traditional credit scoring methods often fail to capture complex, non-linear relationships between borrower characteristics and default risk. By applying data mining and machine learning techniques to 148,670 historical loan records, we developed predictive models capable of identifying high-risk borrowers.

1.2 Dataset Overview (Brief)

- **Source:** Kaggle - Loan Default Prediction dataset (Nikhil E9, 2022)
- **Size:**
  - **Full dataset:** 148,670 loan records with 34 features.
  - **Working subset:** 5,000 samples (stratified sampling for efficient development).
- **Target Variable:** Status: 0 = Non-default (75.36%), 1 = Default (24.64%).
- **Feature Categories:**
  - **Borrower Demographics (5):** Age, gender, dependents, occupancy type, residential status.
  - **Credit Characteristics (6):** Credit scores (primary/co-applicant), credit type, debt-to-income ratio (dtir1).
  - **Loan Structure (10):** Amount, term, purpose, type, interest rates, upfront charges, payment terms.
  - **Property Information (4):** Property value, loan-to-value ratio (LTV), construction type, security.
  - **Application Metadata (9):** Submission channel, region, application year, administrative details.

1.3 Objectives & Tasks

Primary Objectives:

1. Conduct comprehensive exploratory data analysis to understand distributions and relationships.
2. Implement logistic regression from scratch using NumPy to demonstrate algorithm understanding.
3. Train and evaluate multiple models (baseline and advanced) for performance comparison.
4. Use appropriate metrics for cost-sensitive, imbalanced classification.

Completed Tasks:

- Handled 26.66% missing data through median imputation and indicator flags.
- Identified and removed data leakage features (rate_of_interest, interest_rate_spread, upfront_charges).
- Standardized categorical variables and converted data types.
- Created stratified train/validation/test splits (70-15-15).
- Performed univariate and bivariate analyses.
- Implemented complete logistic regression from scratch with gradient descent.
- Trained six models: Scratch LR, Sklearn LR, Random Forest, Gradient Boosting, XGBoost, SVM.
- Evaluated using accuracy, precision, recall, F1-score, and ROC-AUC.
- Extracted and interpreted feature importance rankings.

---

2. Dataset and Preprocessing

2.1 Dataset Details

- **Size and Scope:** 148,670 records × 34 features (working subset: 5,000 samples).
  - **Target:** Status (0 = Non-default: 75.36%, 1 = Default: 24.64%).
- **Feature Types:**
  - **Categorical Features:** Gender, loan_type, loan_purpose, Region, co-applicant_credit_type (Require encoding transformations for machine learning algorithms).
  - **Numerical Features:**
    - loan_amount: $5,000 - $5,000,000+ (heavily right-skewed).
    - income: Annual borrower income (wide range with outliers).
    - Credit_Score: FICO scale 300-850 (roughly normal distribution).
    - LTV: Loan-to-Value ratio 0-100% (concentrated 80-90%).
    - dtir1: Debt-to-Income ratio 5-70% (right-skewed).
    - property_value: Asset backing for secured loans.
- **Data Quality Issues Identified:**
  - Type inconsistencies (numerical data stored as strings).
  - Cryptic categorical codes ("p1", "type1") requiring investigation.
  - Substantial missing data (>26% in some columns).
  - Potential data leakage from post-approval features.

2.2 Preprocessing Steps

Feature Selection: We identified and removed three features causing data leakage:

- rate_of_interest: Determined after risk assessment, not before.
- interest_rate_spread: Reflects lender's risk premium (post-decision).
- upfront_charges: Calculated based on approved loan structure.
- **Impact:** Removing these dropped model performance from perfect scores (1.000) to realistic levels (0.73-0.89), confirming leakage diagnosis.

Missing Values Treatment:

Feature	Missing %	Strategy
Upfront_charges	26.66%	Median + indicator flag
Interest_rate_spread	24.64%	Median + indicator flag
Rate_of_interest	24.51%	Median + indicator flag

Feature	Missing %	Strategy
dtir1	16.22%	Median + indicator flag
Property_value	10.16%	Median imputation
LTV	10.16%	Median imputation
Income	6.15%	Median imputation

Approach:

1. Median imputation for numerical features (robust to outliers in financial data).
2. Mode imputation for categorical features (most frequent category).
3. Missing indicator flags for features >15% missing (captures information content).

Pattern discovered: Property_value and LTV have identical missingness (10.16%), suggesting systematic patterns for unsecured loans.

Outlier Removal:

- Interest rates >12% capped at 99th percentile.
- Loan amounts >$3M capped at 99th percentile.
- Prevents extreme values from distorting gradient-based models.

Normalization:

- StandardScaler: Transformed all numerical features to mean $=0$, $std=1$.
- **Why?** Gradient descent optimization requires features on similar scales.
- Log transformation: Applied to loan_amount due to heavy right-skew.

Feature Encoding:

- **Label Encoding:** Binary features (Gender: Male/Female →0/1).
- **One-Hot Encoding:** Nominal categories (Region, loan_purpose) → dummy variables.
- **Ordinal Encoding:** Preserved for naturally ordered features.

Train-Test Split:

- **Strategy:** Stratified 70-15-15 split.
    - Training: 3,500 samples (70%)
    - Validation: 750 samples (15%)
    - Test: 750 samples (15%)
- **Critical:** All transformations fit ONLY on training data to prevent leakage.

---

3. Algorithm Description

3.1 Description of Algorithm Implemented from Scratch

We implemented Logistic Regression from first principles using only NumPy to demonstrate deep understanding of the underlying mathematics.

Mathematical Foundation:

1. Sigmoid Function:

$$\sigma(z) = 1/(1 + e^{-z})$$

where z = w*X + b

   - Maps any real number to probability range [0, 1].
   - w: weight vector, b: bias, X: input features.

2. Binary Cross-Entropy Loss:

$$J(w, b) = -\frac{1}{m}\sum[y.\log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y})]$$

   - Derived from maximum likelihood estimation.
   - Penalizes confident wrong predictions heavily.

3. Gradient Computation:

$$\frac{\partial J}{\partial w} = \frac{1}{m}X^T(\hat{y} - y)$$

$$\frac{\partial J}{\partial b} = \frac{1}{m}\sum(\hat{y} - y)$$

   - Obtained via chain rule of calculus.
   - Indicates direction to adjust weights.

4. Parameter Update (Gradient Descent):

$$w := w - \alpha\left(\frac{\partial J}{\partial w}\right)$$

$$b := b - \alpha\left(\frac{\partial J}{\partial b}\right)$$

   - α = learning rate (step size).

Implementation Details:

- **Class Structure:** LogisticRegressionScratch
- **Hyperparameters:**
  - Learning rate: 0.01 (determined through experimentation).
  - Max iterations: 1000.
  - Convergence: Loss change < 1e-4 over 20 iterations.

Training Process:

1. **Initialize weights:** Using small random values (np.random.randn(n_features) * 0.01).
2. **Initialize bias:** Set to zero.
3. **For each iteration:**
   - **Forward pass:** Compute predictions $\hat{y} = \sigma(Xw + b)$.
   - **Loss calculation:** Compute binary cross-entropy.
   - **Backward pass:** Calculate gradients.
   - **Update:** Apply gradient descent step.
4. **Stop:** When converged (~300 iterations).

Validation:

- Compared against scikit-learn's LogisticRegression.
- Weight correlation: r > 0.99.
- Prediction correlation: r = 1.00.
- **Conclusion:** Mathematical implementation verified correct.

3.2 Description of Other Models

We trained five additional models for comprehensive performance comparison:

1. **Logistic Regression (Sklearn)**
   - **Purpose:** Validation baseline for our scratch implementation.
   - **Solver:** LBFGS (optimized quasi-Newton method).
   - **Configuration:** max_iter=1000, default parameters.
2. **Random Forest Classifier**
   - **Type:** Ensemble method using bagging (bootstrap aggregating).
   - Builds 100 decision trees on random feature subsets.
   - Each tree votes on final prediction, reducing variance.
   - **Configuration:** n_estimators=100, max_depth=20, class_weight='balanced'.
3. **Gradient Boosting Classifier**
   - **Type:** Sequential ensemble where each tree corrects previous errors.
   - More prone to overfitting than Random Forest but often superior performance.

    - **Configuration:** n_estimators=100, learning_rate=0.1, max_depth=5, subsample=0.8.
4. **XGBoost (Extreme Gradient Boosting)**
    - **Type:** Optimized distributed gradient boosting with built-in regularization.
    - Industry standard for structured data competitions.
    - Uses second-order derivatives for more accurate approximations.
    - **Configuration:** use_label_encoder=False, eval_metric='logloss', random_state=42.
5. **Support Vector Machine (SVM)**
    - **Type:** Finds optimal hyperplane maximizing margin between classes.
    - Effective in high-dimensional spaces but computationally expensive.
    - **Configuration:** kernel='rbf', C=1.0, gamma='scale', class_weight='balanced'.

4. Experiments and Results

4.1 Experiment Setup

- **Train/Test Split:**
    - Training: 3,500 samples (70%)
    - Validation: 750 samples (15%)
    - Test: 750 samples (15%)
    - **Stratification:** Maintained 75.36%/24.64% class distribution.
- **Cross-Validation:** Not performed (limitation noted in Conclusions), single train-test split used.
- **Hardware/Software:**
    - **Platform:** Google Colab (GPU-accelerated for XGBoost).
    - **Language:** Python 3.9.
    - **Libraries:** scikit-learn 1.0, XGBoost 1.6, pandas 1.3, NumPy 1.21.

4.2 Evaluation Metrics

Given class imbalance (24.64% defaults), we employed multiple metrics:

- **Accuracy:** Overall % correct (baseline: 75.36% by predicting all non-default).
- **Precision:** Of predicted defaults, what % actually defaulted? Minimizes false alarms.
- **Recall (Sensitivity):** Of actual defaults, what % did we catch? Minimizes missed risks.
- **F1-Score:** Harmonic mean of Precision and Recall = $2 \cdot (P \cdot R) / (P + R)$.
- **ROC-AUC:** Area under ROC curve - measures separation ability across all thresholds.
    - 0.5 = random guessing.
    - 1.0 = perfect separation.

- Primary metric for imbalanced problems.

4.3 Results of Implemented Model and Other Models

Test Set Performance (750 samples):

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Random Forest	0.8653	0.9151	0.5132	0.6576	0.8869
Gradient Boosting	0.8760	0.8810	0.5873	0.7048	0.8722
XGBoost	0.8680	0.8261	0.6032	0.6972	0.8672
SVM	0.7680	0.5904	0.2593	0.3603	0.7734
LR (Sklearn)	0.7933	0.7125	0.3016	0.4238	0.7330
LR (Scratch)	0.7733	0.7021	0.1746	0.2797	0.7358

Confusion Matrix - Best Model (Gradient Boosting):

	Predicted Non-Default	Predicted Default
Actual Non-Default	546	15
Actual Default	78	111

- **True Positives:** 111 (correctly identified defaults)
- **False Negatives:** 78 (missed defaults)
- **False Positives:** 15 (false alarms)
- **True Negatives:** 546 (correctly identified non-defaults)

Feature Importance (Random Forest):

Rank	Feature	Importance	Interpretation
1	Credit_Score	0.187	Historical repayment reliability
2	LTV	0.142	Borrower equity stake
3	income	0.095	Repayment capacity
4	loan_amount	0.089	Absolute exposure
5	business_or_commercial	0.082	Elevated business risk
6	dtir1	0.053	Debt burden measure
7	property_value	0.047	Asset backing

Logistic Regression Coefficients (Scratch Implementation): Top predictors based on absolute weight: co-applicant_credit_type (+0.39), lump_sum_payment (-0.37), Neg_ammortization (-0.25), Gender_Joint (-0.21), submission_of_application (+0.15).

4.4 Discussion of Results

Key Findings:

1. **Ensemble Dominance:** Tree-based models (RF, GB, XGBoost) achieved ROC-AUC of 0.87-0.89, outperforming linear models by 15-18% (0.73-0.77). This indicates loan default has non-linear decision boundaries and complex feature interactions (e.g., high LTV + low income = compounded risk).
2. **Scratch Implementation Validation:**
    - Our custom LR: ROC-AUC = 0.7358
    - Sklearn LR: ROC-AUC = 0.7330
    - Difference: 0.0028 (negligible). Conclusion: Mathematical derivation and coding verified correct.
3. **The Recall Challenge:** Linear models had 17-30% recall (missing 70-83% of defaults), while XGBoost achieved 60% recall. Higher recall = fewer missed defaults = millions in prevented losses.

4. **Gradient Boosting Optimal for Deployment:** Best F1-score (0.7048), balanced precision (88%) and recall (59%), and strong ROC-AUC (0.8722). It catches 59% of defaults while maintaining 88% precision.
5. **Data Leakage Resolution:** Initial perfect scores (1.000) from rate_of_interest were reduced to realistic scores (0.87-0.89) after removal, confirming leakage diagnosis and ensuring deployability.

Threshold Optimization Analysis:

Threshold	Precision	Recall	F1	Strategy
0.3	0.4815	0.7460	0.5857	Aggressive risk management
0.4	0.6923	0.6508	0.6709	Balanced (optimal F1)
0.5	0.8810	0.5873	0.7048	Conservative lending

Recommendation: Threshold = 0.4 provides best balance for most institutions.

---

5. Conclusions

5.1 Summarize What Was Achieved

Successful Deliverables:

- **Comprehensive Data Pipeline:** Handled 26.66% missing data through robust imputation strategies with indicator flags.
- **Data Leakage Resolution:** Identified and removed three leakage features causing perfect scores, ensuring realistic performance estimates.
- **Scratch Algorithm Implementation:** Built complete Logistic Regression with gradient descent, validated against scikit-learn ($r=1.00$ correlation).
- **Model Benchmarking:** Trained six diverse models achieving ROC-AUC up to 0.8869.
- **Feature Interpretation:** Confirmed Credit_Score, LTV, and income as top predictors, aligning with financial domain expertise.
- **Production-Ready Model:** Gradient Boosting model ready for deployment with balanced precision-recall profile.

5.2 Findings from the Project

- **Finding 1: Non-Linear Decision Boundaries:** Ensemble methods outperformed linear models by 15-18% in ROC-AUC. Loan default risk involves complex feature interactions (e.g., High LTV becomes particularly dangerous when combined with low income).
- **Finding 2: Credit Score Dominance:** Strongest predictor across all models (18.7% importance). 73-point mean difference between defaulters (642) and non-defaulters (715) validates continued relevance of traditional FICO scoring.
- **Finding 3: Business Loan Risk Premium:** Business/commercial loans (38.5% default rate) have 74% higher risk than residential mortgages (22.1% default rate), justifying premium pricing and higher capital reserves.
- **Finding 4: Submission Channel Matters:** Institutional channels (21.7% default rate) outperform individual submissions (29.2% default rate) by 26%, suggesting value in channel partnerships and pre-screening.
- **Finding 5: Recall-Precision Tradeoff:** Best recall was 60.32% (XGBoost), still missing 40% of defaults. This represents a fundamental limitation of predictive modeling, though even 60% recall translates to millions in prevented losses.

5.3 Limitations and Possible Extensions

Current Limitations:

1. **Single Train-Test Split:** Vulnerable to selection bias; different random splits could produce slightly different results; no confidence intervals.
2. **No Cross-Validation:** Relied on single validation set rather than k-fold CV, reducing confidence in estimates and risking overfitting.
3. **Limited Hyperparameter Tuning:** Used reasonable defaults; grid search or Bayesian optimization could improve 2-5%.
4. **Class Imbalance Techniques Not Applied:** Did not experiment with SMOTE or ADASYN to improve minority class recall.
5. **Small Test Set:** 750 samples (only ~185 actual defaults) creates substantial confidence interval width.
6. **No Feature Engineering:** Did not create interaction terms (e.g., Credit_Score × LTV) or polynomial features.
7. **Temporal Validation Absent:** Dataset lacks temporal structure, preventing validation of generalization over time.

Possible Extensions:

- **Immediate Next Steps:** Implement 5-fold cross-validation, apply SMOTE/ADASYN, perform grid search tuning, create interaction features, and expand test set.
- **Advanced Extensions:** Implement SHAP values for interpretability, neural network architectures, ensemble stacking, cost-sensitive learning, and a deployment pipeline.

6. References

Datasets:

1. Nikhil E9 (2022). Loan Default Prediction Dataset. Kaggle. https://www.kaggle.com/datasets/nikhil1e9/loan-default

Papers:

2. Hastie, T., Tibshirani, R., & Friedman, J. (2009). The Elements of Statistical Learning: Data Mining, Inference, and Prediction (2nd ed.). Springer.

3. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of KDD 2016, 785-794.

Libraries Used:

4. Scikit-learn: Pedregosa et al. (2011). Machine Learning in Python. JMLR 12, 2825-2830. https://scikit-learn.org/stable/

5. Pandas: McKinney, W. (2010). Data Structures for Statistical Computing in Python.

6. NumPy: Harris, C. R., et al. (2020). Array programming with NumPy. Nature 585, 357-362.

7. XGBoost Documentation: https://xgboost.readthedocs.io/

8. Matplotlib/Seaborn: Hunter, J. D. (2007). Matplotlib: A 2D graphics environment.

Course Materials:

9. EECS 4412 Data Mining lecture notes, tutorials, and assignments. York University, Fall 2025-2026.

AI Tools Consulted:

10. ChatGPT (OpenAI) - Used for code debugging, syntax suggestions, documentation structure, and explanation refinement.

11. GitHub Copilot - Used for code autocompletion and debugging assistance.

7. Appendices

Appendix A: GitHub Link

- **Repository URL:** [Insert your GitHub repository link here]
- **Repository Structure:**
  - data/: Loan_default.csv
  - notebooks/: 01_EDA.ipynb, 02_Preprocessing.ipynb, 03_Scratch_LR.ipynb, 04_Model_Comparison.ipynb
  - src/: preprocessing.py, logistic_regression.py, evaluation.py
  - models/: gradient_boosting.pkl
  - reports/: README.md

Appendix B: User's Manual

- Step 1: Installation - Clone repository and install dependencies: pip install pandas numpy scikit-learn xgboost matplotlib seaborn
- Step 2: Prepare Data - Place Loan_default.csv in data/ directory.
- Step 3: Run Preprocessing - Execute notebooks/02_Preprocessing.ipynb.
- Step 4: Train Models - Execute notebooks/04_Model_Comparison.ipynb.
- Step 5: Make Predictions - Use load_model and predict from src.evaluation.

Appendix C: System Design

- **Components:** Data Ingestion, Preprocessing (Imputer, Encoder, Scaler), Model Training (6 classifiers), Evaluation.
- **Data Structures:** Pandas DataFrames, NumPy arrays.
- **Handling Large Datasets:** Vectorized operations, chunked processing capability, memory-efficient types.

Appendix D: Sample Input and Output

- **Input:** CSV with borrower details (Age, Income, LoanAmount, etc.)
- **Output:** JSON with default probability, risk category, recommendation, and risk factors.

Appendix E: Program Limitations and Known Bugs

- **Limitations:** Max dataset size (~500k rows without chunking), strict feature requirements, prediction latency (~500ms), memory overhead from one-hot encoding, manual retraining.
- **Bugs:** Log transform edge case ($0 loan amount), categorical value mismatch in test data, NumPy overflow warning for very large amounts.

Appendix F: Contribution Statement

- **Devarsh Prajapati (20%):** LogisticRegressionScratch implementation, validation, learning curves.
- **Pratham Khatri (20%):** Preprocessing pipeline, leakage resolution, missing value imputation, video presentation.
- **Vedansh Patel (20%):** Experimental setup, evaluation metrics, threshold optimization, results interpretation, report synthesis.
- **Neel Shethji (20%):** Visualization suite, EDA, report formatting, missing value imputation.
- **Rithik Chaudhary (20%):** Ensemble models integration, hyperparameter tuning, feature importance, power point presentation.